

1. feladat – Pick & Place: lépésről lépésre, szkriptek nélkül

Ez az útmutató végigvezet az 1. mérési feladaton úgy, hogy **semmilyen szkriptre vagy programozásra nincs szükség**.

- Mindent a RoboDK menüiből csinálsz.
- A mérési eredményeket a RoboDK kijelzőiről olvasod le: az állapotsorból és a robotpanelből.
- Ami nem olvasható le közvetlenül, azt egyszerű képletekkel számolod ki (4. fejezet).
- A dokumentációt Wordben írod meg a kész sablonba: [jegyzokonyv_sablon.docx](#). Ebben minden fejezet és mérési táblázat előre el van készítve.

Mire lesz szükséged: RoboDK, Microsoft Word (vagy LibreOffice Writer), számológép vagy Excel, és a Windows képernyőkép-készítője (**Win+Shift+S**).

Időigény:

- az előkészület kb. 30 perc;
- a cella és a programok felépítése 1–2 óra;
- a mérések kb. 1 óra;
- a jegyzőkönyv befejezése 1–2 óra.

A feladat röviden. Egy modern, 6 tengelyes robotkar vegyen fel egy munkadarabot, és tegye át egy másik helyre, movej és move'l mozgással, különböző sebességekkel.

Mérendő adatok:

1. a felvételtől a lerakásig eltelt idő;
2. a mozgás során megtett út;
3. a robot maximális sebessége;
4. a movej és a move'l végrehajtási idejének különbsége.

Sikeres a feladat, ha a robot ütközés nélkül, pontosan és ismételhető módon helyezi át a munkadarabot.

Tartalom

- 1. Előkészületek
- 2. A megoldás terve
- 3. A cella és a programok felépítése lépésről lépésre
- 4. Mérési módszerek szkript nélkül
- 5. Mérések lépésről lépésre
- 6. Számítási példák
- 7. Az eredmények értelmezése
- 8. A jegyzőkönyv befejezése Wordben
- 9. Gyakori hibák
- 10. Mit írsz az értékelésbe?

1. Előkészületek

1.1 Programok és beállítások

1. **RoboDK:** robodk.com/download. Ha az egyetemnek van licence, azt használd. Különben 30 napig próbaverzió fut, utána a mentés korlátozott lehet. Egyetemi e-mail-címmel a RoboDK-tól kérhetsz hosszabbítást.
2. **Nyelv:** az útmutató az angol menüneveket használja. Ha a RoboDK más nyelven indul, állítsd át: **Tools → Language → English**.
3. **Word-sablon:** töltsd le a [jegyzokonyv_sablon.docx](#) fájlt, nyisd meg, és mentsd el a saját nevedre (pl. Feladat1_jegyzokonyv.docx). Ebbe dolgozol végig.
4. **Képernyőkép:** **Win+Shift+S** → jelöld ki a területet → a Wordben **Ctrl+V**. Windows 11-en a kép a *Képek\Képernyőképek* mappába is elmentődik.
5. **Mappa:** hozz létre egy mappát (pl. *C:\RoboDK_feladatok*). Ide kerül a *Feladat1.rdk* állomás és a jegyzőkönyv.

1.2 RoboDK-alapok (5 perc gyakorlás)

Mit akarsz	Hogyan
Kijelölés	bal egérgomb
Nézet forgatása / eltolása / nagyítása	jobb gombot nyomva tartva húzd / középső gombot (görgőt) nyomva tartva húzd / görgő
Keret vagy objektum mozgatása egérrel	Alt + húzás
A robot TCP-jének mozgatása egérrel	Alt + Shift + húzás
Robotpanel megnyitása	dupla kattintás a robotra. Itt látod a csuklósögeket (<i>Joint axis jog</i>) és a TCP helyzetét a kiválasztott referencia kerethez képest (<i>Tool ... w.r.t. Reference: X, Y, Z és a forgatások</i>)
Robot pontos helyzetbe állítása	a robotpanelen írd be az X, Y, Z (és forgatás) értékeket, majd Enter
Célpont (target) felvétele	Program → Teach Target (Ctrl+T): a robot aktuális helyzetét rögzíti az aktív referencia keretben
A robot célpontra állítása	kattints a célpontra a fában: a robot odaáll
Célpont koordinátái	jelöld ki → F3 (Target Options)
Átnevezés	jelöld ki → F2
Program futtatása	dupla kattintás a programra. A végén az alsó állapotsorban megjelenik a program ideje és a pálya hossza
Egyetlen utasítás végrehajtása	dupla kattintás az utasításra
Szimuláció: szünet / gyorsítás / leállítás	Backspace / Szókőz nyomva tartva / Esc
Szimuláció visszajátszása	futtatáskor alul megjelenik egy csúszka : ezzel előre-hátra tekerheted a mozgást
A TCP pályájának kirajzolása	Tools → Trace
Pálya ellenőrzése / pálya + ütközés ellenőrzése	jobb klikk a programra → Check path (F5) / Check path and collisions (Shift+F5)
Mentés	Ctrl+S

Laptonon, egér nélkül: jobb klikk a 3D nézetben, ott is megtalálod a forgatás, eltolás és nagyítás parancsokat.

1.3 Dokumentálás menet közben

A jegyzőkönyv legfontosabb része a munka menetének bemutatása. Ezt **menet közben** érdemes megírni:

- Ahol az útmutatóban 📸 jelet látsz, készíts képernyőképet (**Win+Shift+S**), és illeszd be a Word-sablon A *megvalósítás lépései* fejezetébe, a lépés helyére.
- A kép alá írd egy mondatot arról, mit csináltál, milyen értékekkel. Például: „Az Asztal referencia keret létrehozása a robot bázisához képest $X = 400\text{ mm}$ -re.”
- Képaláírás: **Hivatkozás → Képaláírás beszúrása**.
- A mért értékeket **azonnal** írd be a sablon táblázataiba, és a leolvasásról is készíts képet (állapotsor, robotpanel). Ez a mérés bizonyítéka.

2. A megoldás terve

Robot: Universal Robots UR5e. Ez modern, 6 tengelyes kollaboratív robot, és a feladatban szereplő „movej” és „movel” is UR-kifejezés.

Minden koordináta az **Asztal** referencia keretben értendő. A szerszám lefelé néz.

Elem	Típus	Adatok
Megfogo	szerszám	TCP: Z = 160 mm a karimától (a munkadarab közepénél fog)
Asztal	referencia keret	a robot bázisához képest X = 400, Y = 0, Z = 0 mm
Asztallap	doboz	500 × 800 × 20 mm, hely: (0; 0; -20), szürke
Munkadarab	doboz	50 × 50 × 50 mm, hely: (0; -200; 0), kék
Lerakohely	doboz (jelölő)	70 × 70 × 1 mm, hely: (0; 200; 0), zöld

A „hely” a doboz **alaplapjának közepe**.

Célpont	X	Y	Z	Megjegyzés
Home	-50	0	300	kiinduló és végpont
Felvetel_felett	0	-200	125	100 mm-rel a megfogási pont fölött
Felvetel	0	-200	25	a kocka közepe (50 mm magas kocka)
Lerakas_felett	0	200	125	
Lerakas	0	200	25	

A lefelé néző szerszám a kerethez képest 180°-kal van elforgatva az X tengely körül. A UR5e robotpanelén ez **Rx = 180**, **Ry = 0**, **Rz = 0**. Ellenőrzés: a 3D nézetben a szerszám kék (Z) tengelye lefelé, az asztal felé mutasson.

Sebességszint	lineáris [mm/s]	lin. gyorsulás [mm/s ²]	csukló [°/s]	csuklógyorsulás [°/s ²]
lassú	100	250	30	60
közepes	300	800	90	180
gyors	800	2000	180	400

Programok:

- PP_MoveJ_lassu, PP_MoveJ_kozepes, PP_MoveJ_gyors
- PP_MoveL_lassu, PP_MoveL_kozepes, PP_MoveL_gyors

A két változat csak a szabad mozgásokban különbözik: Home → felvétel fölé, átvitel, vissza Home-ba. A megközelítés és az emelés mindig MoveL, egyenesen függőlegesen.

Mérőprogramok: Meres_atvitel_J és Meres_atvitel_L. Csak a felvételtől a lerakásig tartó részt tartalmazzák, így annak ideje és útja külön mérhető (4.2).

3. A cella és a programok felépítése lépésről lépésre

1. **Új állomás:** **File** → **New Station**, majd azonnal **File** → **Save Station as...** → **Feladat1.rdk**.
2. **Robot:** **File** → **Open Robot Library** (Ctrl+Shift+O; régebbi verzióban *Open online library*). Keress rá: **UR5e**, majd **Open**. A robot megjelenik az állomásban.  „A UR5e robot betöltése a RoboDK könyvtárából.”
3. **Szerszám (megfogó):**
 - **a) Valódi megfogóval:** a könyvtárban keress egy megfogót (pl. *Robotiq 2F-85*), **Open**. Ha nem a robotra került, a fában húzd rá a robotra.
 - **b) Egyszerűen:** jelöld ki a robotot, majd **Program** → **Add Empty Tool**. Kattints duplán az új szerszámmra, a TCP-nél add meg a **Z = 160** értéket, és nevezd át (F2) **Megfogo**-ra.

 „Megfogó hozzáadása, TCP: Z = 160 mm.”
4. **Referencia keret:** jelöld ki a fában a robot bázisát (*UR5e Base*), majd **Program** → **Add Reference Frame**. Nevezd át (F2) **Asztal**-ra, kattints rá duplán, és állítsd be: **X = 400, Y = 0, Z = 0**, a forgatások 0.
5. **Dobozok: asztallap, munkadarab, lerakóhely.** Három lehetőség van:
 - **A) Components add-in (ajánlott, pontos méretekkel).**
 1. **Utilities** → **Components** → **Shape** → a listában **Box**.
 2. Add meg a méreteket (X, Y, Z) és a szint, majd hozd létre.
 3. Ha nincs ilyen menüpontod, az add-in nincs bekapcsolva. **Tools** → **Add-in Manager** (Shift+A): kapcsold be a *Components* (régebbi neve: *Shape*) add-int. Ha nincs a listában, telepítsd a [Components Add-in oldaláról](#).
 4. Ha az *Add-in Manager* sem látszik: **Tools** → **Add-ins** (Shift+I) → kapcsold be.
 - **B) Online könyvtár:** Ctrl+Shift+O → keress rá: *box*. Méretezés: dupla kattintás az objektumra → **More options** → **Apply Scale**.
 - **C) Saját STL-fájl:** bármilyen kocka **STL File** → **Open**-nel betölthető.

Mindhárom dobozt húzd a fában az **Asztal** keret alá. Kattints rájuk duplán, és add meg a helyüket a 2. fejezet táblázata szerint. Nevezd át őket (F2): **Asztallap**, **Munkadarab**, **Lerakohely**.

Hol van a doboz origója? A táblázatban a „hely” a doboz alaplapjának közepe. Ellenőrzés:

1. Állítsd a doboz helyét (0; 0; 0)-ra.
2. Nézd meg, hol van az **Asztal** keret origója a dobozhoz képest.

Ha az origó a doboz egyik sarkában van, X-ből és Y-ből vond le a méret felét:

- Munkadarab : (-25; -225; 0);
- Asztallap : (-250; -400; -20);
- Lerakohely : (-35; 165; 0).

 „Az Asztal keret, az asztallap, a munkadarab és a lerakóhely elhelyezése.”

6. Célpontok:

1. Kattints duplán a robotra. A robotpanel tetején az aktív szerszám legyen **Megfogo**, az aktív referencia **Asztal**.
2. Minden célpontonál írd be a koordinátákat a 2. fejezet táblázatából a **Tool ... w.r.t. Reference** mezőkbe (Rx = 180, Ry = 0, Rz = 0), majd Enter: a robot odaáll.
3. **Program** → **Teach Target** (Ctrl+T), majd nevezd át (F2).

Sorrend: **Home**, **Felvetel_felett**, **Felvetel**, **Lerakas_felett**, **Lerakas**. Ha a robot nem tud odaállni, a pont nem elérhető. Ilyenkor tedd közelebb az **Asztal** keretet (pl. X = 350).  „A célpontok felvétele.”

Közben töltsd ki a sablon T1.0 táblázatát. Kattints sorban a célpontokra, és írd be a robotpanelről a csuklósögeket (J1–J6). A MoveJ-számításhoz (6.2) ezekre lesz szükséged.

7. Az első program (**PP_MoveJ_kozepes**): jelöld ki a robotot, **Program → Add Program**, és nevezd át (F2). Az utasításokat ebben a sorrendben add hozzá. Az új utasítás mindig a kijelölt után kerül.

#	Menü	Beállítás
1	Program → Simulation Event Instruction	Set object position (relative) : jelöld ki a Munkadarab -ot a kiinduló helyén, OK. Minden futás elején visszateszi a kockát.
2	Program → Set Speed Instruction	lineáris 300 mm/s, lin. gyorsulás 800 mm/s ² , csukló 90 °/s, csuklógyorsulás 180 °/s ² (mindegyiket pipáld be)
3	jelöld ki a Home célpontot → Program → Move Joint Instruction	
4	Felvetel_felett → Move Joint Instruction	
5	Felvetel → Move Linear Instruction	függőleges megközelítés
6	Program → Pause Instruction	300 ms (a megfogó zárási ideje)
7	Program → Simulation Event Instruction	Attach object , szerszám: Megfogo . Pipáld be a <i>Check shortest distance between TCP and the object shape</i> opciót, hogy biztosan a kockát fogja meg, ne az asztallapot.
8	Felvetel_felett → Move Linear Instruction	emelés
9	Lerakas_felett → Move Joint Instruction	átvitel
10	Lerakas → Move Linear Instruction	
11	Program → Pause Instruction	300 ms
12	Program → Simulation Event Instruction	Detach object : szerszám Megfogo , a darab az Asztal keretre kerül
13	Lerakas_felett → Move Linear Instruction	
14	Home → Move Joint Instruction	

Az első mozgásutasításnál a RoboDK magától beszúrja a referencia keret és a szerszám kiválasztását (*Set Reference Frame*, *Set Tool Frame*): ezek maradjanak. Ha egy mozgásutasítás rossz célpontozhoz kötődött: jobb klikk az utasításon → *Target linked* lista.

Kattints duplán a programra: a robot átteszi a kockát. 📡 „A PP_MoveJ_kozepes program: megközelítés, megfogás, átvitel, elengedés.”

8. Ütközésvizsgálat beállítása:

- Tools → Check collisions:** be. Ütközéskor a program megáll, és az ütköző elemek pirosak lesznek.
- Tools → Collision map** (Shift+X): itt állítod be, mely párokat vizsgálja (dupla kattintás egy cellára: be/ki). Alapból minden mozgó elempárt vizsgál, ezért kapcsold ki a szándékosan érintkező párokat:
 - Megfogo - Munkadarab ;
 - Munkadarab - Asztallap ;
 - Munkadarab - Lerakohely .
- A robot tagjai és a megfogó az asztallappal maradjanak bekapcsolva: ez a valódi ütközésveszély.
- Jobb klikk a programra → **Check path and collisions** (Shift+F5).

📡 „Ütközésvizsgálat: a program ütközésmentes.”

9. Sebességváltozatok:

- Jelöld ki a programot → **Ctrl+C**, jelöld ki az állomást → **Ctrl+V**, majd nevezd át (F2) PP_MoveJ_lassu -ra.
- Kattints duplán a *Set Speed* utasításra, és írd át a lassú értékekre.

Ugyanígy készül a PP_MoveJ_gyors .

10. MoveL-változatok:

1. Másold a PP_MoveJ_kozepes -t → PP_MoveL_kozepes .
2. A 4., 9. és 14. utasításon (a szabad mozgásokon): jobb klikk → váltsd lineárisra (*Linear Move*).

Ebből is készíts lassú és gyors másolatot. 📷 „A hat program: MoveJ és MoveL, három sebességgel.”

11. Mérőprogramok (a felvételtől a lerakásig tartó rész méréséhez):

- **Meres_atvitel_J :**
 1. **Program → Add Program**, majd **Set Speed Instruction** (közepes értékek).
 2. Jelöld ki a Felvetel_felett célpontot → **Move Linear Instruction**.
 3. Lerakas_felett → **Move Joint Instruction**.
 4. Lerakas → **Move Linear Instruction**.
- **Meres_atvitel_L :** ugyanez, de a középső mozgás is *Move Linear*.

📷 „Mérőprogramok a felvételtől a lerakásig tartó rész méréséhez.”

12. Mentés: Ctrl+S.

4. Mérési módszerek szkript nélkül

A feladat összes mérendő adatát az alábbi módszerekkel kapod meg. Az 5. fejezet hivatkozik rájuk.

4.1 Ciklusidő és pályahossz – az állapotsorból

1. **Állítsd a robotot a program első célpontjára:** kattints a fában arra a célpontra, amelyre a program első mozgása vezet (a PP_... programoknál Home). Ha ezt kihagyod, a RoboDK az aktuális helyzetből odavezető mozgást is beleszámolja.
2. **Kattints duplán a programra,** és várd meg a végét. A **Szóköz** nyomva tartásával gyorsíthatsz, ez a kiírt időt nem változtatja meg.
3. A program végén **az alsó állapotsorban** megjelenik a **program ideje** (*program time / cycle time*) és a **pálya hossza** (*path length*). A pontos szöveg RoboDK-verzióként kicsit eltérhet.
4. **Futtasd le még egyszer, és a második eredményt jegyezd fel.** Az első futás még a korábbi sebesség- és lekerekítés-beállításokkal számolhat.
5.  Készíts képet az állapotsorról.

Ne stopperrel mérj! A RoboDK alapból 5-ször gyorsabban szimulál a valós időnél, és a megjelenítés is késleltet. Az állapotsorban kiírt idő a valódi robotra becsült idő, ezt kell használni.

4.2 Részidő (felvételtől lerakásig) – mérőprogrammal

Az állapotsor mindig a teljes program idejét mutatja. A felvételtől a lerakásig tartó részhez ezért a mérőprogramokat használd:

1. Kattints a **Felvétel** célpontra: a robot a felvételi pontba áll.
2. Futtasd a mérőprogramot (4.1): az állapotsor a felvételtől a lerakásig tartó mozgás idejét és útját mutatja.
3. A sebességet a mérőprogram **Set Speed** utasításában állítod át (dupla kattintás rá).

4.3 Helyzet és csuklószőgek – a robotpanelből

1. Kattints duplán a robotra. A panel tetején az aktív szerszám legyen **Megfogo**, az aktív referencia keret **Asztal**.
2. A **Joint axis jog** részben a csuklószőgek (J1...J6) láthatók. A mellettük lévő másolás gombbal kimásolhatod őket (pl. Excelbe).
3. A **Tool ... w.r.t. Reference** mezőkben a TCP helyzete látható az **Asztal** keretben (X, Y, Z és a forgatások).
4. **Egy célpontban** úgy olvasol le, hogy rákattintasz a célpontra, vagy duplán kattintasz a programban arra a mozgásutasításra, amely odavezet.
5. **Mozgás közben** úgy olvasol le, hogy megállítod a szimulációt (**Backspace**), vagy a csúszkával a kívánt pillanathoz tekersz.

4.4 Idő és sebesség kiszámítása (trapéz sebességprofil)

A RoboDK dokumentációja szerint a robot egyenletesen gyorsul a beállított sebességig, majd egyenletesen lassul. Egy pontos (megálló, lekerekítés nélküli) mozgás ideje ezért kézzel is kiszámolható:

L = a mozgás hossza: MoveL-nél a két célpont távolsága [mm],
MoveJ-nél a legnagyobb csuklóelfordulás $|\Delta J|$ [$^{\circ}$]
 v, a = a beállított sebesség és gyorsulás: MoveL-nél a lineáris [mm/s, mm/s²],
MoveJ-nél a csukló [$^{\circ}$ /s, $^{\circ}$ /s²] értékek
 $s_{gy} = v^2 / a$ (ennyi út kell a felgyorsuláshoz és a megálláshoz együtt)

ha $L \geq s_{gy}$: $t = L / v + v / a$ és a robot eléri a beállított v sebességet
ha $L < s_{gy}$: $t = 2 \cdot \sqrt{L / a}$ és a legnagyobb sebesség csak $v_{max} = \sqrt{a \cdot L}$

- Megálló célpontoknál a szakaszok ideje **összeadódik**. A várakozások (Pause) ideje is hozzáadódik.
- MoveJ-nél minden csukló egyszerre indul és egyszerre ér célba, ezért a **legnagyobb elfordulású csukló** határozza meg az időt.

Ha a RoboDK által kiírt idő nagyon eltér a számítottól:

- ellenőrizd, hogy a *Set Speed* minden értéke be van-e pipálva;
- nézd meg a **Tools → Options → Motion → Move time calculation** beállítást. Alapértelmezés szerint a MoveJ a csukló-, a MoveL a lineáris értékekkel számol.

4.5 Távolság és út kiszámítása

Két pont távolsága:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

A MoveL egyenes vonalon halad, ezért útja a célpontok közötti távolságok összege. A MoveJ útja ennél hosszabb, mert a TCP ívben mozog. A MoveJ útját az állapotsor *path length* értékéből kapod meg (4.1).

4.6 Pontosság (eltérés a kívánt helyzettől)

1. Olvasd le a TCP helyzetét a célpontban (4.3).
2. Hasonlítsd össze a célpont koordinátaival (F3).
3. Az eltérés: $e = \sqrt{(\Delta X^2 + \Delta Y^2 + \Delta Z^2)}$.

A lerakás pontosságát a munkadarabon is ellenőrizheted:

1. A program után kattints duplán a **Munkadarab**-ra.
2. Olvasd le a helyét az **Asztal** keretben.
3. Hasonlítsd össze a lerakóhellyel.

5. Mérések lépésről lépésre

A sablon *Mérési eredmények* fejezetében minden méréshez van táblázat: T1.1–T1.6. A célpontok csuklósögei a T1.0-ban vannak (3. fejezet, 6. lépés).

1. Ciklusidő és teljes pályahossz → T1.1. Mind a 6 PP_... programnál:

1. Kattints a **Home** célpontra.
2. Futtasd a programot kétszer.
3. Írd fel a második futás idejét és pályahosszát az állapotsorból (4.1). 📷

2. Idő a felvételtől a lerakásig → T1.2:

1. Kattints a **Felvetel** célpontra, és futtasd a **Meres_atvitel_J**-t (4.2). Írd fel az időt és az utat.
2. Írd át a mérőprogram **Set Speed** utasítását lassúra, majd gyorsra, és mérj újra.
3. Ugyanezt csináld meg a **Meres_atvitel_L**-lel.

Ez 6 mérés. Mindegyik mellé írd a számított időt is (4.4; a kiszámolt értékek a 6. fejezetben vannak).

3. A megtett út → T1.1 és T1.2:

- A teljes program útja az állapotsor *path length* értéke.
- A felvételtől a lerakásig tartó MoveL-út kézzel: $100 + 400 + 100 = 600 \text{ mm}$ (fel, át, le). Hasonlítsd össze a mérttel.
- A MoveJ-út ennél hosszabb, mert ívelt.

4. A robot legnagyobb sebessége → T1.3 (4.4):

- **MoveL**: számold ki szakaszonként a v_{max} -ot. Elég hosszú-e a függőleges 100 mm-es és a vízszintes 400 mm-es szakasz ahhoz, hogy a robot elérje a beállított sebességet?
- **MoveJ**: keresd meg a T1.0-ban a **Felvetel_felett** és a **Lerakas_felett** közötti legnagyobb $|\Delta J|$ -t. Ebből kiszámolod a csukló legnagyobb szögsebességét.
- **Átlagsebesség**: út / idő, a mérőprogramok méréseiből.
- **Adatlap**: írd mellé a robot adatlap szerinti legnagyobb sebességét is. Az UR5e-nél ez kb. 1 m/s TCP-sebesség és 180 °/s csuklósebesség; ellenőrizd az adatlapon.

5. A MoveJ és a MoveL idejének különbsége → T1.4. Sebességszintenként, a T1.2 méréseiből:

- $\Delta t = t(\text{MoveL}) - t(\text{MoveJ})$
- $\Delta\% = \Delta t / t(\text{MoveL}) \cdot 100$

Ugyanezt a teljes programokra is számold ki (T1.1).

6. Pontosság és ismételhetőség → T1.5:

- **Pontosság**: állítsd a robotot a **Felvetel**, majd a **Lerakas** célpontra, és olvasd le a TCP helyzetét (4.6).
- **A munkadarab helye**: futtasd a **PP_MoveJ_kozepes** programot, majd kattints duplán a **Munkadarab**-ra, és olvasd le a helyét. Ha a doboz origója az alaplap közepén van, a várt érték (0; 200; 0). Sarokpontú dobozban ez a kiinduló hely plusz 400 mm Y irányban.
- **Ismételhetőség**: futtasd a programot még kétszer. A program eleje mindig visszateszi a kockát. Minden futás után írd fel a ciklusidőt és a kocka helyét.

7. Ütközésmentesség → T1.6. Mind a 6 programra: jobb klikk → **Check path and collisions** (Shift+F5), majd írd fel az eredményt. 📷

6. Számítási példák

6.1 Felvételtől lerakásig, MoveL

Szakasz	L [mm]	lassú (100 mm/s; 250 mm/s ²)	közepes (300; 800)	gyors (800; 2000)
$s_{gy} = v^2/a$		40 mm	112,5 mm	320 mm
emelés Felvetel → Felvetel_felett	100	1,400 s (eléri a 100 mm/s-ot)	0,707 s ($v_{max} = 282,8$ mm/s)	0,447 s ($v_{max} = 447,2$ mm/s)
átvitel → Lerakas_felett	400	4,400 s	1,708 s	0,900 s (eléri a 800 mm/s-ot)
leengedés → Lerakas	100	1,400 s	0,707 s	0,447 s
összesen	600	7,20 s	3,12 s	1,79 s

Részletesen a közepes sebességnél:

- 400 mm:** $s_{gy} = 300^2/800 = 112,5$ mm, ez kisebb, mint 400 mm, tehát a robot eléri a 300 mm/s-ot. $t = 400/300 + 300/800 = 1,333 + 0,375 = 1,708$ s.
- 100 mm:** $100 < 112,5$, tehát a robot nem éri el a 300 mm/s-ot. $t = 2 \cdot \sqrt{(100/800)} = 0,707$ s, és $v_{max} = \sqrt{(800 \cdot 100)} = 282,8$ mm/s.

Gyors beállításnál a 100 mm-es függőleges szakaszokon a robot nem éri el a 800 mm/s-ot, mert a gyorsulás korlátozza: $s_{gy} = 320$ mm > 100 mm. Ezért a **nyolcszoros** sebesség (100 → 800 mm/s) csak **kb. négyszer** rövidebb időt ad (7,20 s → 1,79 s).

6.2 Felvételtől lerakásig, MoveJ

A függőleges emelés és leengedés itt is MoveL (6.1). Csak az átvitel MoveJ, ennek idejét a legnagyobb csuklóelfordulás adja. Ezt a saját T1.0 táblázatodból olvasd ki.

Az UR5e-nél ez várhatóan a **J1** (a talp forgása) és a **J6** (a csukló forgása, amely megtartja a megfogó irányát). A két célpont a talphoz képest ± 200 mm-re van oldalt, 400 mm-re előre, így $|\Delta J1| \approx 2 \cdot \arctan(200/400) \approx 53,1^\circ$.

	lassú (30 °/s; 60 °/s ²)	közepes (90; 180)	gyors (180; 400)
$s_{gy} = \omega^2/\epsilon$	15°	45°	81°
átvitel MoveJ, $ \Delta J = 53,1^\circ$	$53,1/30 + 30/60 = 2,27$ s	$53,1/90 + 90/180 = 1,09$ s	$2 \cdot \sqrt{(53,1/400)} = 0,73$ s ($\omega_{max} = 146$ °/s)
+ emelés és leengedés (MoveL, 6.1)	$2 \times 1,400$ s	$2 \times 0,707$ s	$2 \times 0,447$ s
összesen	5,07 s	2,50 s	1,62 s

6.3 A MoveJ és a MoveL összehasonlítása (számítás alapján)

Sebességszint	t(MoveL)	t(MoveJ)	Δt	$\Delta\%$
lassú	7,20 s	5,07 s	2,13 s	29,6 %
közepes	3,12 s	2,50 s	0,62 s	19,8 %
gyors	1,79 s	1,62 s	0,17 s	9,5 %

Ezek számított értékek. A jegyzőkönyvbe a **saját mért értékeid** kerülnek (T1.2, T1.4), és mellettük ezek a számítások mint ellenőrzés. Ha a mért és a számított érték eltér, keresd meg az okát (4.4).

7. Az eredmények értelmezése

7.1 Mit mér a szimuláció, és mit nem?

- A RoboDK **ideális kinematikai modellel** számol. A **megálló (pontos, „fine”) célpontokat** a robot **sebességtől függetlenül pontosan eléri**: az eltérés 0 mm a kijelzés felbontásán belül. A szimuláció **determinisztikus**, ugyanaz a program mindig ugyanazt adja. Az ismételhetőség tehát szimulációban ideális.
- **Valódi robotnál** a nagyobb sebesség és gyorsulás nagyobb pályahibát okoz: a hajtások késnek, a robot túllendül, a karok rugalmasan deformálódnak. A pozicionálási ismételhetőséget az adatlap adja meg (ISO 9283); az UR5e-nél ez $\pm 0,03$ mm. Ellenőrizd az adatlapon.
- A **ciklusidő** a RoboDK becslése a beállított sebességek és gyorsulások alapján (4.4). A valódi robotnál ettől kissé eltérhet.

7.2 Várható eredmények

- **MoveJ vs. MoveL:**
 - A MoveJ **gyorsabb**, mert minden csukló egyszerre, a saját csuklósebességével mozog. A TCP pályája ilyenkor ív, ezért **hosszabb utat** tesz meg.
 - A MoveL-nél a TCP egyenesen halad a beállított lineáris sebességgel: **rövidebb az út**, de hosszabb az idő.
 - A különbség nagy sebességnél kisebb (6.3), mert ott a gyorsítás és a lassítás dominál.
- **Sebesség:** a kétszeres sebesség nem jelent fele ciklusidőt. A gyorsítás és a lassítás (4.4), valamint a rögzített várakozások (2×300 ms) nem rövidülnek arányosan.
- **Legnagyobb sebesség:** a beállított sebességet a robot csak a hosszú szakaszokon éri el. A rövid, 100 mm-es függőleges szakaszokon gyors beállításnál csak kb. 447 mm/s-ig gyorsul (6.1).
- **Pontosság, ismételhetőség:** a TCP a célpontokban pontosan a kívánt helyen van, a kocka pontosan a lerakóhelyre kerül, és három futás után is ugyanaz az eredmény. A szimulációban ez 0 mm eltérés (7.1).
- **Ütközés:** a helyesen beállított ütközésvizsgálat egyik programnál sem jelez ütközést.

8. A jegyzőkönyv befejezése Wordben

A `jegyzokonyv_sablon.docx` felépítése:

1. Címlap
2. Tartalomjegyzék
3. Bevezetés, a mérési módszerrel és korlátaival
4. A feladat leírása (előre kitöltve)
5. Az állomás felépítése
6. Célpontok (T1.0)
7. Programok
8. A megvalósítás lépései: ide kerülnek a 📸 képernyőképeid
9. Mérési eredmények (T1.1–T1.6)
10. Értékelés
11. Összefoglalás
12. Források

Teendők:

- A sárga, szögletes zárójeles részeket ([...]) cseréld ki a saját szövegedre, adataidra és képeidre, majd töröld a kiemelést.
- A táblázatok **halványsárga cellába** írd a mért és a számított értékeket. Ha kész vagy, a cellák színét visszaállíthatod fehérre (Táblázattervezés → Mintázat → Nincs szín).
- **Tartalomjegyzék:** jobb klikk → **Mező frissítése** (F9), ekkor bekerülnek az oldalszámok.
- **Képaláírás:** Hivatkozás → Képaláírás beszúrása.
- **Képlet:** Beszúrás → Egyenlet (Alt+=), vagy egyszerűen írd le szöveggént.
- **PDF:** Fájl → Mentés másként → PDF.
- **Ajánlott melléklet:** jobb klikk egy programra → **Export Simulation** → 3D HTML. Ezt az oktató a böngészőben körbe tudja forgatni.

8.1 Ellenőrzőlista beadás előtt

- ☐ A `Feladat1.rdk` el van mentve, és mind a 6 program hibátlanul fut (F5, Shift+F5).
- ☐ A *megvalósítás lépései* fejezetben minden 📸 lépésnél van kép és egy-két mondat.
- ☐ A T1.0–T1.6 táblázatok ki vannak töltve, és a leolvasásokról is van kép (állapotsor, robotpanel).
- ☐ A számított és a mért idők egymás mellett szerepelnek, és megmagyaráztad az eltérést, ha van.
- ☐ Nincs több sárga [...] rész a Word-fájlban.
- ☐ Az értékelésben a saját mért számaid szerepelnek, és megmagyaráztad a szimuláció korlátait (7.1).
- ☐ A tartalomjegyzék frissítve van, és a PDF elkészült.

9. Gyakori hibák

Hiba	Megoldás
A robot nem éri el a pontot (a robotpanelen nem mozdul, vagy hibát jelez)	Tedd közelebb az <code>Asztal</code> keretet a robothoz (X csökkentése), vagy emeld a pontokat.
A MoveL-re szingularitást vagy csuklóhatárt jelez	A szabad mozgásnál cseréld MoveJ-re (jobb klikk → Joint Move), vagy változtass a célpont helyén.

Hiba	Megoldás
Furcsa, túl hosszú ciklusidőt mutat	A robot nem a program első célpontjáról indult (kattints rá előtte), vagy ez volt az első futás (futtasd újra).
A számított és a mért idő eltér	Ellenőrizd a Set Speed értékeit (mindegyik be van-e pipálva), és a Tools → Options → Motion → Move time calculation beállítást.
A kocka nem mozdul a megfogóval, vagy az asztallapot fogja meg	Hiányzik az Attach object esemény, vagy túl messze van a TCP a kockától (alapból 200 mm a tűrés). Pipáld be a Check shortest distance between TCP and the object shape opciót.
A második futtatásnál a kocka rossz helyről indul	A program elejére tegyél Simulation Event → Set object position (relative) utasítást (3. fejezet, 7. lépés 1. sora).
Ütközést jelez, pedig csak érintés van (megfogó és kocka, kocka és asztal)	Tools → Collision map: kapcsold ki az adott párt (3. fejezet, 8. lépés).
Nincs Utilities → Components menü	Kapcsold be vagy telepítsd a Components add-int, vagy használd a könyvtár dobozait (3. fejezet, 5. lépés).
A doboz rossz helyen van	Valószínűleg a sarkában van az origója. Lásd a 3. fejezet 5. lépésének megjegyzését.
Nem látszik az állapotsor üzenete	Az üzenet a program végén jelenik meg az ablak alján. Ha közben mást csinálsz, eltűnhet: futtasd újra a programot.

10. Mit írj az értékelésbe?

Az értékelés legyen a **te mérésed** értelmezése. Az alábbi kérdésekre válaszolj, a jegyzőkönyv táblázataira hivatkozva (pl. „a T1.2 táblázat szerint...”).

1. Mennyi volt a felvételtől a lerakásig eltelt idő a három sebességnél? Arányosan csökkent-e? Ha nem, miért (gyorsítás, lassítás, várakozások)? Egyezik-e a számított idővel?
2. Melyik volt gyorsabb, a MoveJ vagy a MoveL, és mennyivel (s, %)? Melyik tett meg rövidebb utat? Miért? Hogyan változott a különbség a sebességgel?
3. Mekkora volt a robot legnagyobb sebessége? Elérte-e a beállított sebességet a rövid, 100 mm-es szakaszokon?
4. Teljesül-e a sikerességi feltétel (ütközés nélkül, pontosan, ismételhetően)? Mit mutatnak ehhez a mérések, és mi lenne más valódi robotnál?

Hasznos RoboDK-dokumentáció

- [Basic Guide](#): égerkezelés, könyvtár, gyorsbillentyűk
- [Getting Started](#): állomás, robot, szerszám, célpont, program
- [A Simple Pick and Place Example](#): a RoboDK saját pick & place példája
- [Robot panel](#): csuklószogek és TCP-helyzet leolvasása
- [Program instructions](#): Set Speed, Pause, mozgásutasítások
- [Simulation event](#): Attach/Detach object, Set object position
- [Simulate Program](#): futtatás, csúszka, lépésenkénti végrehajtás
- [Cycle Time](#): hogyan számolja a RoboDK a ciklusidőt
- [Tools menu](#): Trace, Check collisions, Measure
- [Collision Detection](#) és [Check path \(F5\)](#)
- [Components Add-in](#): doboz létrehozása megadott méretekkel